Nyaya Saathi: A Multi-Agent Legal Assistant for India

Devansh Verma | Kuku AI Generalist Assignment, Problem 2: Build an AI Lawyer
Prototype: nyaya-saathi.streamlit.app | Code and raw outputs: github.com/myselfdevansh/nyaya-saathi

Abstract. *Most people in India never get help with everyday legal problems. Nyaya Saathi takes a problem by text or voice in English, Hindi or Hinglish, sends it to one of six specialist agents, asks only the questions that change the answer, and replies in about 150 words with what to do and when a lawyer is needed. Legal claims are limited to official text from 49 Central Acts, and code checks every citation before the user sees it. The router placed 26 of 26 test problems correctly. The main weakness is search: it found the right section for only about two-thirds of test questions in the best earlier run, and when it misses, answers get thinner or lean on weaker sections.*

1. Problem choice and scope

I chose Problem 2 because the harm is concrete and the output can be checked: every legal claim can be compared with the official text. Covering all Indian law is not realistic, so I scoped to the everyday problems named in the brief, grouped into six agents: Consumer Rights, Property and Rental, Employment and Labour, Cybercrime and Criminal, Family Law, and Banking and Finance. A general agent handles anything else, such as road accidents or RTI.

2. Assumptions

- Users have a phone and little legal knowledge, and prefer short steps to long explanations.
- Central Acts are the right free starting point. State rent law, rules, notifications and case law are out of scope, and the assistant says so rather than guessing.
- This gives legal information, not advice. It should get people moving and tell them when a lawyer is needed.
- Payment of Wages and Payment of Gratuity Acts had no section records on India Code, so they are excluded.

3. How it works

Message or voice	Triage (code)	Front desk: pick agent, ask questions (LLM)	Search agent's Acts (code)	Answer from sources (LLM)	Citation and limit checks (code)

Fig. 1. One turn of conversation. Code owns safety and verification.

Data. 5,004 sections from 49 Acts come from India Code's public API, split into 6,749 chunks with multilingual embeddings that run on CPU. **Agents.** An agent is one model plus its own Acts, questions, guidance and complaint channels, not a separate model. If a person picks an area, the router doesn't overrule them. **Questions.** Up to 8, one at a time in a pop-up, skipped for danger or arrest. **Language.** Replies follow the user's script. Hindi speech gets about 50-50 Hinglish, or full Devanagari with the Hindi button. **Output.** Where you stand, what to do, what to keep as proof, when to see a lawyer, linked sources, and a draft notice or complaint with placeholders for missing facts. **Model swap.** One line in `.env`; every provider uses the same client, with automatic fallback when a model hits its limit.

Given to the LLM	Kept as code or rules
Routing, clarifying questions, query rewriting, plain-language answers, drafting, speech-to-text	Danger, arrest and fraud triage; language detection; Act scope per agent; citation checks; removing unverifiable ombudsman limits

Table 1. Division of work.

4. How it avoids invented law

- **Closed sources.** The model sees 8 retrieved sections and may cite only their IDs.
- **Citation checker.** Any cited section not retrieved is removed and noted. Free-form and Hindi citations are matched to retrieved sections or removed.
- **Flags.** IPC/CrPC references and case names are flagged, since the system has no case-law database.
- **Scheme limits.** Ombudsman deadlines aren't in the data, so code replaces any such line with a pointer to the official site.
- **Human route.** Urgent banners and lawyer or NALSA 15100 referrals are built in; thumbs up/down feedback is logged.

5. How I know it is good

Check	Result
Agent routing, 26 problems in 3 languages	Gemini 26/26; backup keywords 21/26
Right section in top 8, user's words only, 22 questions	BM25 9%; embeddings 55%; embeddings in keyword-routed agent's Acts 59%
Same, with LLM rewrite (earlier 23-Act version)	64-73%, varying by run
Automated guardrail and UI tests, with a stand-in LLM	11 and 11 pass
Fixed: unverifiable ombudsman limits, 4 real answers	Model stated one in 2; 0 reached the user
Voice, generated Hindi audio	Transcribed correctly in both scripts
End-to-end conversations, 14 in raw outputs	Unedited, with retrieved sources and checker notes

Table 2. Results from real runs. The test sets are small and self-written, so scores are optimistic.

Good examples from the runs: a delayed flat cites RERA s.18 on return of amount and compensation; an elderly parent gets the Senior Citizens Act maintenance route; a Hindi domestic violence message shows 112 and 181 first. Asked for Supreme Court judgments it can't verify, it doesn't supply them.

6. Where it fails

Failure seen	Status
Right law not retrieved. Job scam: BNS s.318 (cheating) wasn't retrieved, the checker removed the citation, and the answer cited weaker forgery sections. Cheque bounce: NI Act s.138 was flagged as unverified.	Open. Search is the top priority.
Thin answers. Unpaid salary cites only Code on Wages s.59 (burden of proof) and misses when wages are due and how to claim.	Open.
Misapplied law. An earlier run applied Industrial Relations Code s.59 (lay-off dues) to ordinary salary. The checker proves a section was retrieved, not that it fits.	Reduced by a prompt rule, not solved. Needs a verifier or human review.
Model ignores instructions. It stated a 30-day RBI wait despite being told not to.	Fixed in code for one-line cases. A limit on a separate line could pass.

Untested. Real microphone speech; languages beyond English and Hindi.	Open.

Table 3. Known failures.

7. Scale and cost

Each message makes 3 model calls (front desk, rewrite, answer), plus 1 for voice. Embeddings run locally and hosting is free, so the model is the main running cost. The free tier is the first limit: one earlier run exhausted a model's quota after about 45 calls, and fallback models only delay that. Real traffic needs a paid API, which needs no code change, plus a real server, because the free host sleeps when idle and takes a minute or more to restart. New law scales cheaply: add an Act's title, rerun the script, rebuild the index (about 10 minutes for 49 Acts on a laptop). State law is the highest-value addition. Paid case-law APIs could help, and I'd use them to improve questions before showing any case to users.

8. Human in the loop

The assistant doesn't replace review. Before real use, a lawyer should sample answers each week, starting with thumbs-down cases, and approve every agent's question list and complaint channels. I'd measure whether people took the next step, not only whether they liked the answer.

9. AI tools, and where they got it wrong

I used Claude Code to build and test the system and Gemini as the app's model. Both made mistakes. Claude told me Hugging Face Docker Spaces were free; the site showed them as paid, and I moved to Streamlit Cloud. Gemini misapplied law and stated the RBI wait above. The lesson shaped the design: check against the source, and keep the rules that matter in code.

10. Next steps

Fix search first, using questions I didn't tune on. Then add a verifier pass and state rent law. Then distil the deciding facts from 20-30 judgments per agent into better questions, using the CC-BY Supreme Court and High Court datasets, and compare the questions before and after.